Deep Reinforcement Learning

Brief Introduction to Deep Learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

Content:

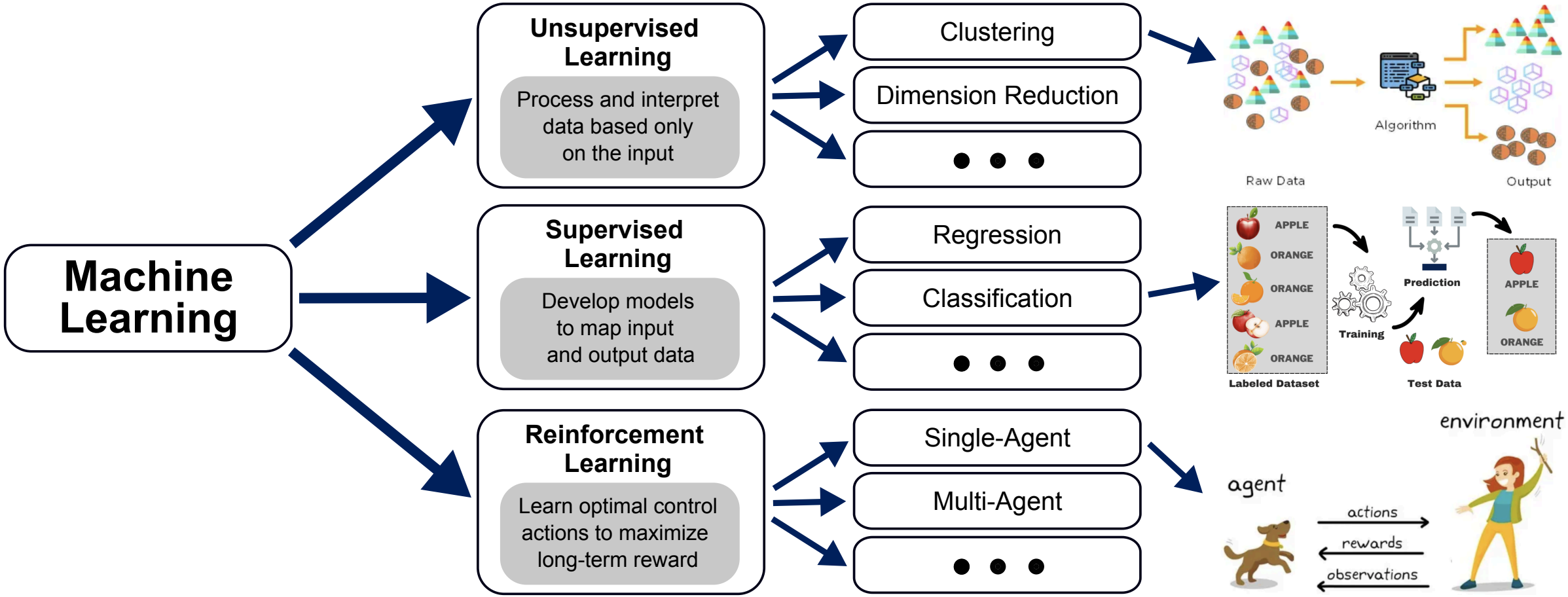
- The three main learning paradigms
- Supervised learning
  - The learning diagram
  - Generalization
  - Bias & variance
  - Cross-validation
- Deep neural networks
  - Artificial neural networks
  - Multi-layer perceptron
  - Training
  - Regularization
  - Neural network architectures
  - Linear regression as a special case

Useful references:

- Pattern recognition and machine learning (Bishop 2006)
- Deep learning (Bengio et al. 2017)
- The elements of statistical learning (Hastie 2009)
- Learning from data: a short course (Abu-Mostafa, Magdon-Ismail, and Lin 2012)

The three main learning paradigms

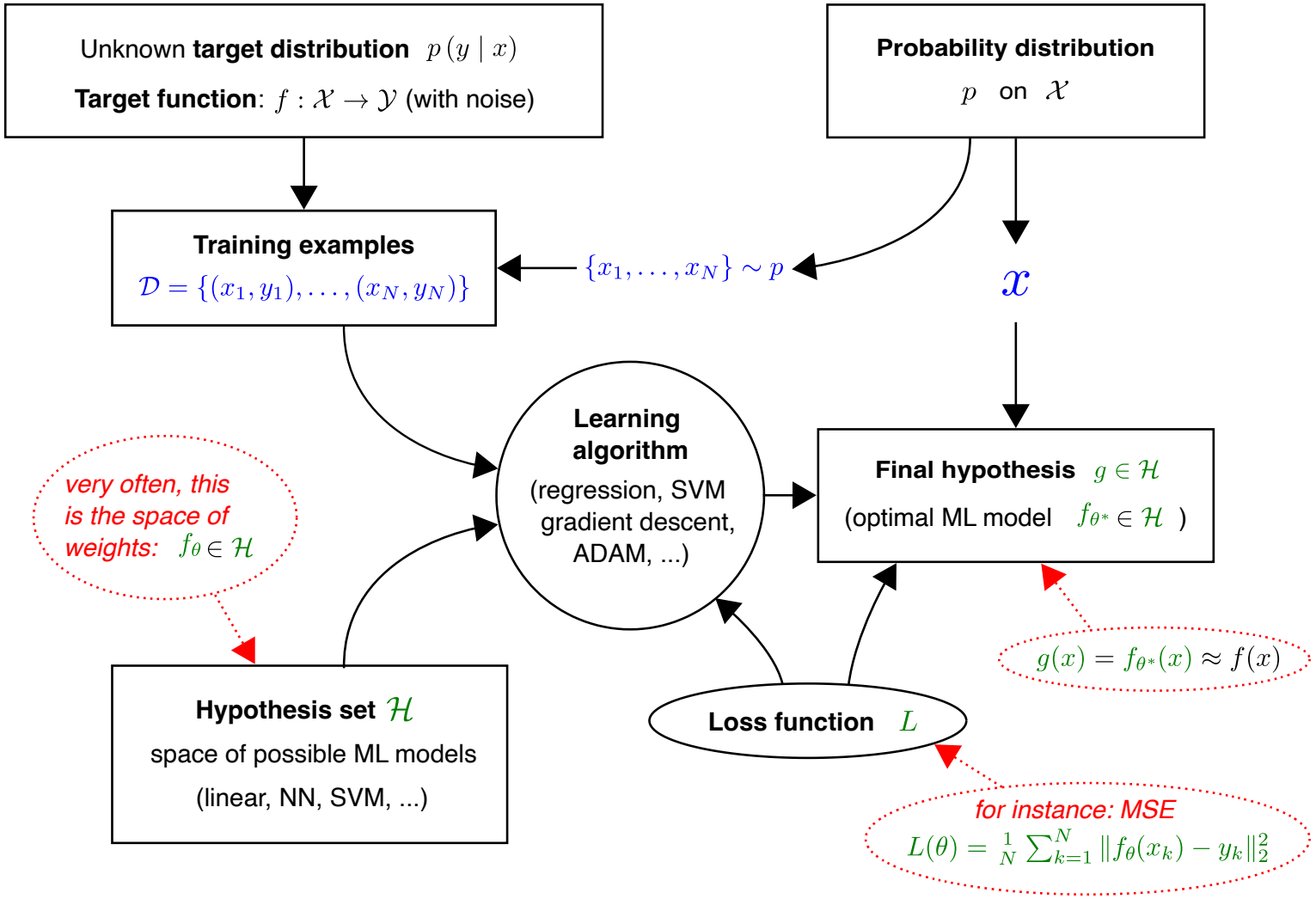
The three main learning paradigms



The three big learning paradigms (Abdelwanis et al. 2026) (Examples: Unsupervised, Supervised, RL)

Supervised learning

The learning diagram



The learning diagram (Abu-Mostafa, Magdon-Ismail, and Lin 2012)

Generalization

- What does it mean to have a perfect model on your training dataset $\mathcal{D}_{\text{train}}$, i.e., $L(w, \mathcal{D}_{\text{train}}) = 0$?
  $\Rightarrow$ we have simply memorized the data!
- Learning means that we get **good predictions on unseen data**:

$$\underbrace{L(w, \mathcal{D}_{\text{test}})}_{\text{in-sample error}} \approx \underbrace{L(w, \mathcal{D}_{\text{train}})}_{\text{out-of-sample error}} .$$

Supervised learning
Given a labeled (and probably noisy) dataset $\mathcal{D}_{\text{train}} = \{(x_1, y_1), \dots, (x_N, y_N)\}$, approximate the unknown mapping $f : \mathcal{X} \to \mathcal{Y}$ by a parametrizable ML model $f_\theta : \mathcal{X} \to \mathcal{Y}$, such that

$$f_\theta(x_k) = \hat{y}_k \approx y_k \quad \forall\, (x_k, y_k) \in \mathcal{D}_{\text{test}}.$$

- **Goodness of fit** can be measured via many different metrics (e.g., mean squared error, classification accuracy, etc.).
- The dimension d of model parameters $\theta \in \mathbb{R}^d$ is adjustable in many model families, which trades off **bias** with **variance** (among other factors, leading to so-called under- and overfitting).

- On top of θ, an ML model might also have **hyperparameters** that can be optimized (e.g., number of layers in a neural network).

Bias-variance tradeoff (1)

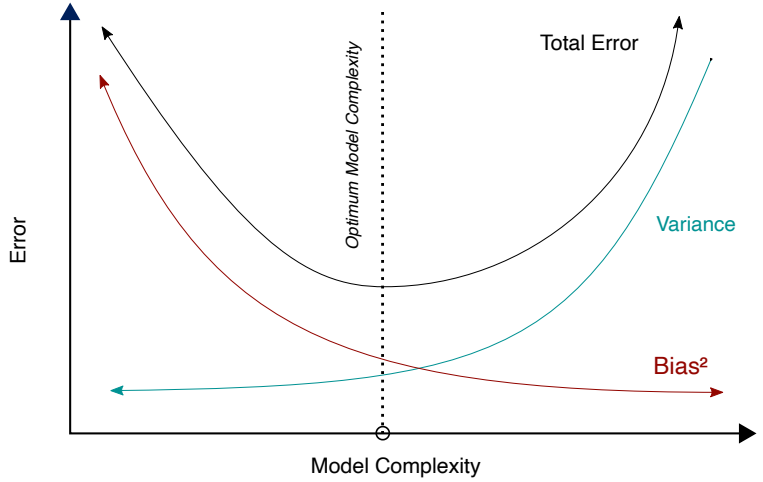
- In the ML context, **bias** denotes the error of the *average* model $\overline{f}_\theta$ when repeating the training with different datasets $\mathcal{D}_{\text{train},1}, \mathcal{D}_{\text{train},2}, \ldots$:

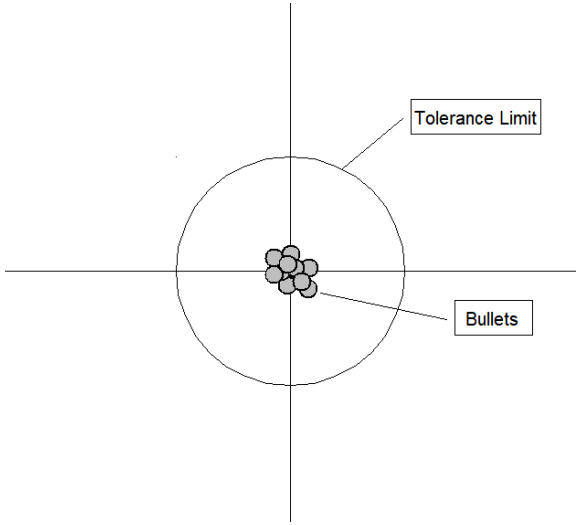
$$\text{bias} = \mathbb{E}_{x \sim \mathcal{D}_{\text{test}}} \left[\left(\overline{f}_\theta(x) - f(x) \right)^2 \right].$$

- **variance** denotes the variability in between the individual training runs:

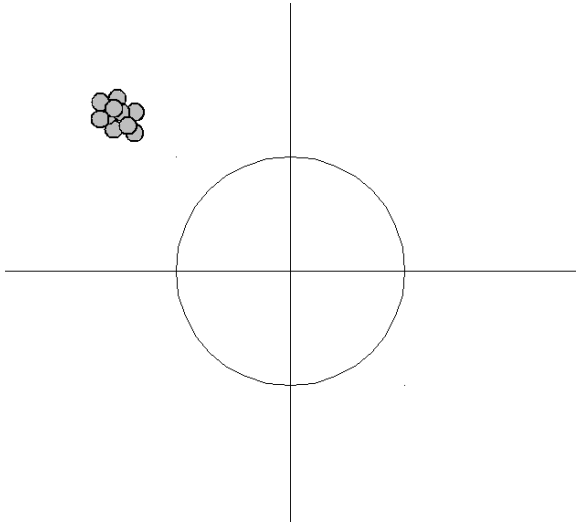
$$\text{variance} = \mathbb{E}_{x \sim \mathcal{D}_{\text{test}}} \left[\mathbb{E}_{\mathcal{D} \sim \{\mathcal{D}_{\text{train},\ell}\}_{\ell=1}^{\infty}} \left[\left(f_\theta^{(\mathcal{D})}(x) - \overline{f}_\theta(x) \right)^2 \right] \right].$$

$\Rightarrow$ Often a matter of model complexity.

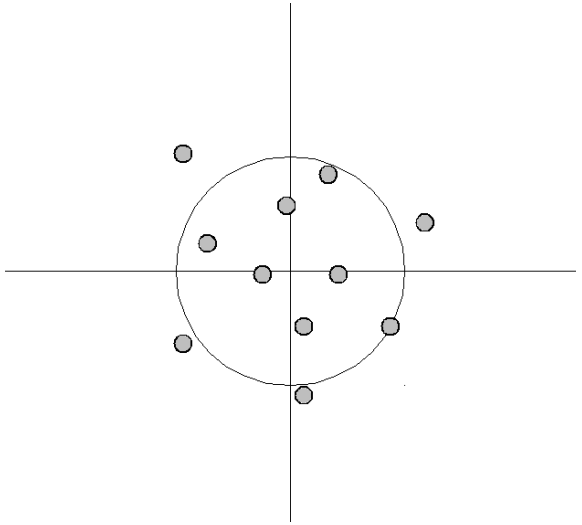




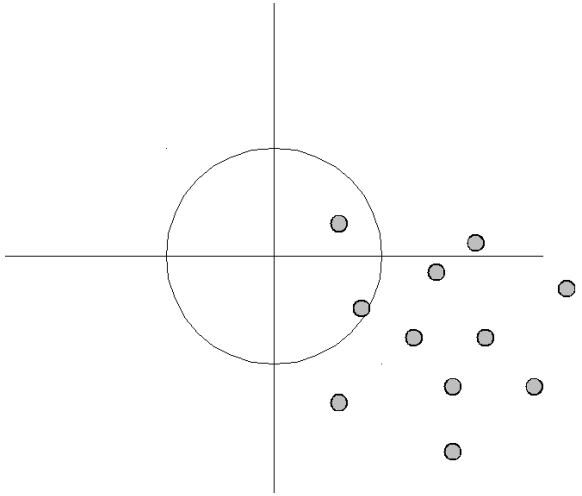
Low bias, low variance



Large bias, low variance



Low bias, large variance



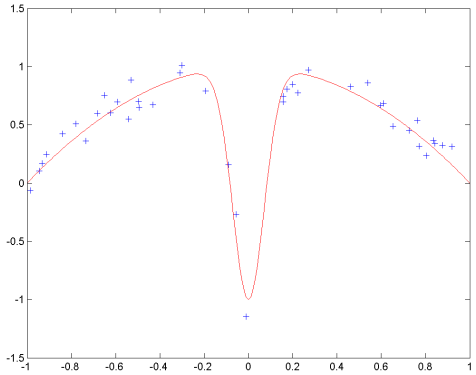
Large bias, large variance [Wikipedia]

Bias-variance tradeoff (2)

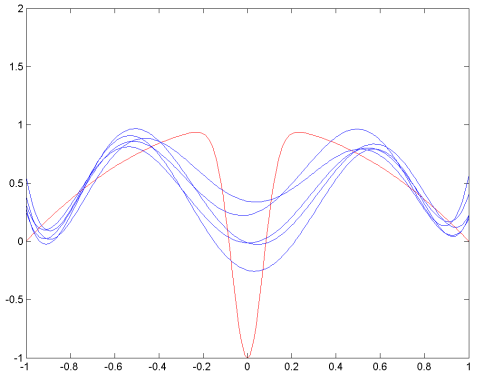
Example (see Wikipedia): Fitting a model of serveral radial basis functions to noisy trainig data:

$$f_\theta(x) = \sum_{k=1}^{d} \theta_k \exp\left(-\frac{1}{2}\frac{x - c_k}{\sigma_k^2}\right).$$

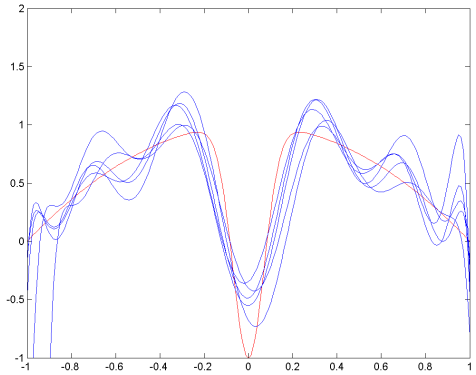
- For a wide spread, the bias is high: the RBFs cannot fully approximate the function (especially the central dip), but the variance between different trials is low.
- As spread decreases (image 3 and 4) the bias decreases: the blue curves more closely approximate the red...
- ... but the variance between trials ($\mathcal{D}_{\text{train},1}, \mathcal{D}_{\text{train},2}, \ldots$) increases.



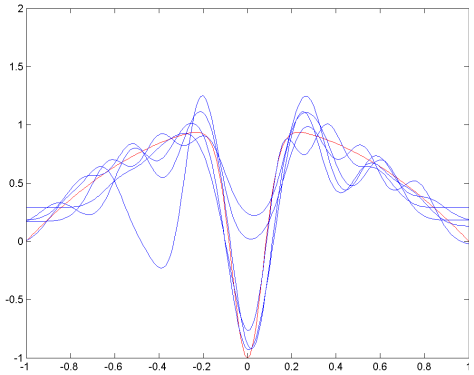
Function and training data $\mathcal{D}_{\text{train},1}$.



Wide spread RBFs.

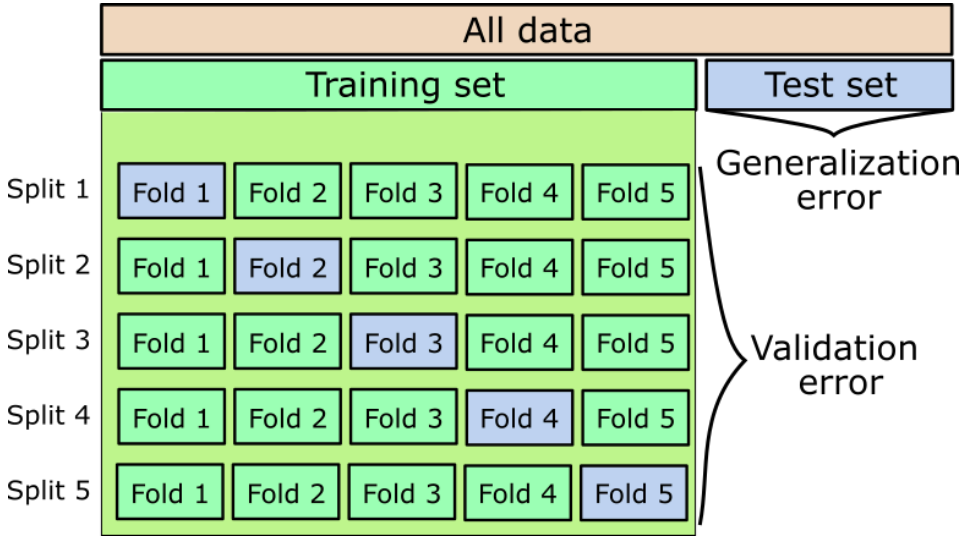


Medium spread RBFs.



Small spread RBFs.

Cross-validation

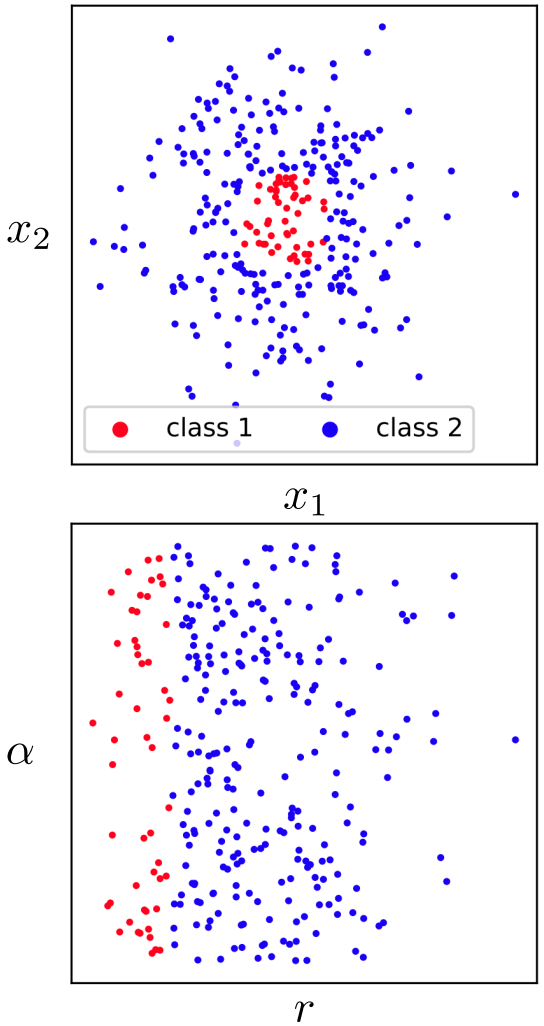


5-fold CV example (Abdelwanis et al. 2026).

- Training is repeated k times with k different splits of the training set.
- Each observation serves as unseen instance (blue boxes) at least once.
- The validation error is an indicator for tuning hyperparameters.
- Example of a k-fold Cross-validation (CV).

Means to improve a supervised learning model

- Collecting **more data**, i.e., increasing N.
- **Reducing noise** in the data.
- **Improving the distribution** within the dataset, i.e., ensuring that the data set is representative of the problem domain.
- **Choosing a more appropriate model**. A genereal rule is to select the model according to the amount of data one has, not according to the expected complexity of the funtion to approximate.
- **Optimizing hyperparameters** of the model.
- **Ensemble learning**: Averaging over several different models.
- **Including knowledge**, e.g., in the form of tailored features (feature engineering) or informed loss functions.



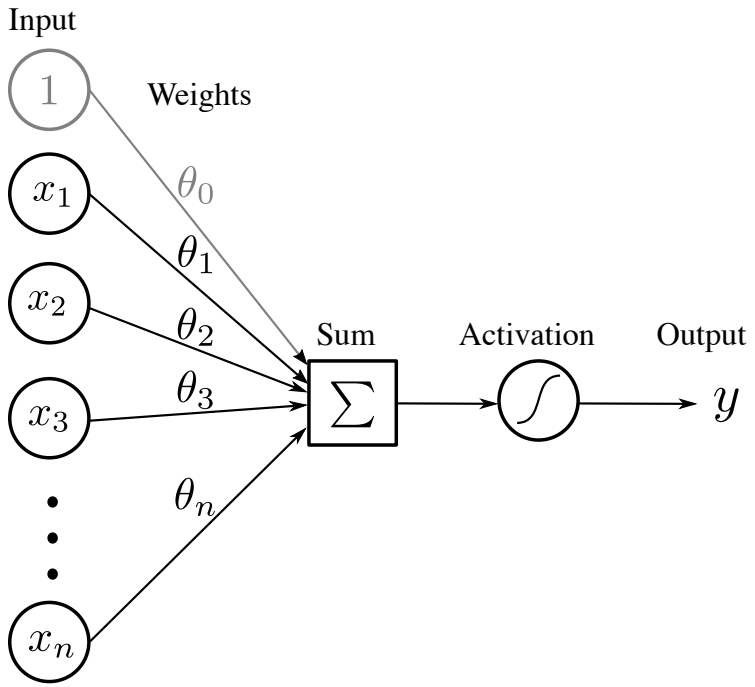
Euclidean vs. polar coordinates/features in binary classification (Abdelwanis et al. 2026).

Deep neural networks

Artificial neural networks

Artificial neural networks (ANNs) are nonlinear function approximators $\hat{y} = f_\theta(x)$ that

- are end-to-end differentiable.
- are stacks of minimal units, the **artificial neurons**.



An artificial neuron (Abdelwanis et al. 2026).

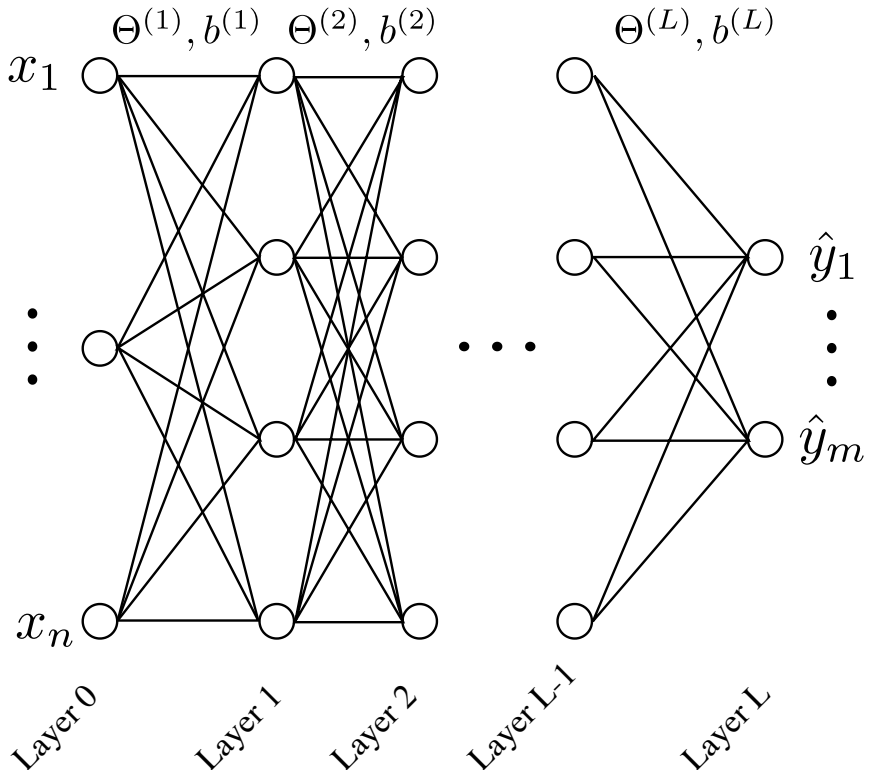
- An ANN consists of **nodes** or **neurons** in one or more layers.
- Each node transforms the weighted sum of all previous nodes (plus a potential **bias term**) through an **activation function** σ:

$$\sigma\left(\theta_0 + \sum_{k=1}^{n} \theta_k x_k\right).$$

- The weighted connections are called **edges**, which represent the ANN's parameters.

Multi-layer perceptron

Standard model of supervised learning: **multi-layer perceptron** or **feed-forward ANN**.



MLP architecture (Abdelwanis et al. 2026).

- Only forward-flowing edges.
- The depth L and width $H^{(\ell)}$ are hyperparameters.
- With $\sigma^{(\ell)}$ and $z^{(\ell)}$ denoting the activation function and **activation** of layer ℓ respectively, we get for the output in the ℓth layer.

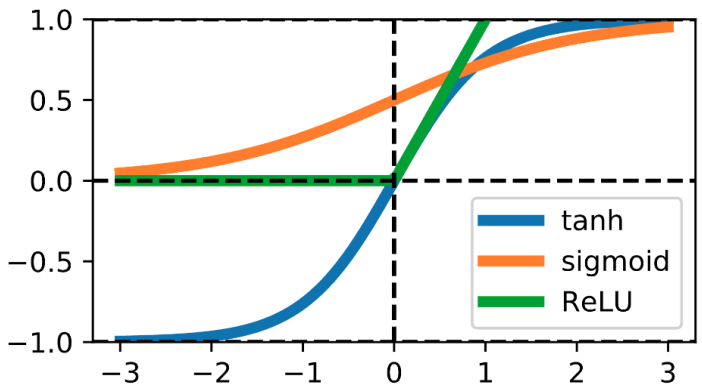
$$x^{(\ell)} = \sigma^{(\ell)}\Big(\underbrace{\Theta^{(\ell)}x^{(\ell-1)}+b^{(\ell)}}_{z^{(\ell)}}\Big),$$

  with input $x^{(0)} = x$ and output $x^{(L)} = y$:

- Training:
  - Summarize the full set of parameters (i.e., weight matrix $\Theta^{(\ell)} \in \mathbb{R}^{H^{(\ell)}\times H^{(\ell-1)}}$ and biases $b^{(\ell)}$) under θ.
  - Iteratively update the weights using gradient information.

Activation functions

- The source of nonlinearity in neural networks.*
- Common choices for $\sigma(z)$ are
  - $\sigma(z) = \tanh(z)$,
  - Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$,
  - Rectified linear unit (ReLU): $\sigma(z) = \max(0, z)$,



Exemplary activation functions (Abdelwanis et al. 2026).

- The activation of the output layer, $\sigma^{(L)}(z)$, is task-dependent. For instance,
  - regression: $y = \sigma^{(L)}(x^{(L-1)}) = x^{(L-1)}$, i.e., $\sigma^{(L)} = \mathsf{Id}$ is the identity mapping.
  - binary classification: sigmoid (i.e., probability), followed by a rounding step to either 0 or 1.
  - multi-class classification: $y_i = \dfrac{\exp(z_i^{(L-1)})}{\sum_j \exp(z_j^{(L-1)})}$ (softmax).

* Without nonlinear activation functions, every ANN collapses to a single matrix-vector multiplication $y = \hat{\Theta}x$:

$$x^{(\ell+2)} = \Theta^{(\ell+2)}x^{(\ell+1)} = \Theta^{(\ell+2)}\left(\Theta^{(\ell+1)}x^{(\ell)}\right) = \left(\Theta^{(\ell+2)}\Theta^{(\ell+1)}\right)x^{(\ell)} = \hat{\Theta}x^{(\ell)}.$$

For non-zero biases, we obtain an *affine* transformation instead.

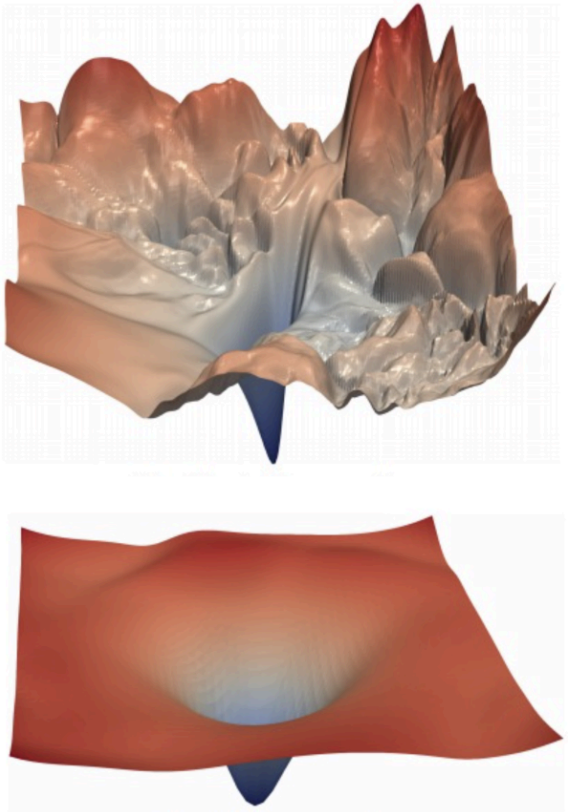
Training (1)

- Training is performed in an iterative manner:

$$\theta \leftarrow \theta + \eta \delta\theta,$$

  where $\eta \in \mathbb{R}_{>0}$ is the **step size** or **learning rate**, and $\delta\theta \in \mathbb{R}^d$ is the **update direction**, usually a gradient-based **descent direction**.

- First, we need to define a **loss function** that we wish to minimize.
  - Regression: (root) mean square error, mean absolute error.
  - Classification: cross entropy.



The loss landscape of deep neural networks (with and without skip-connection) (Li et al. 2018).

Training (2)

- The descent direction is computed by taking the derivative of the loss function w.r.t. the weight vector. In terms of the MSE:

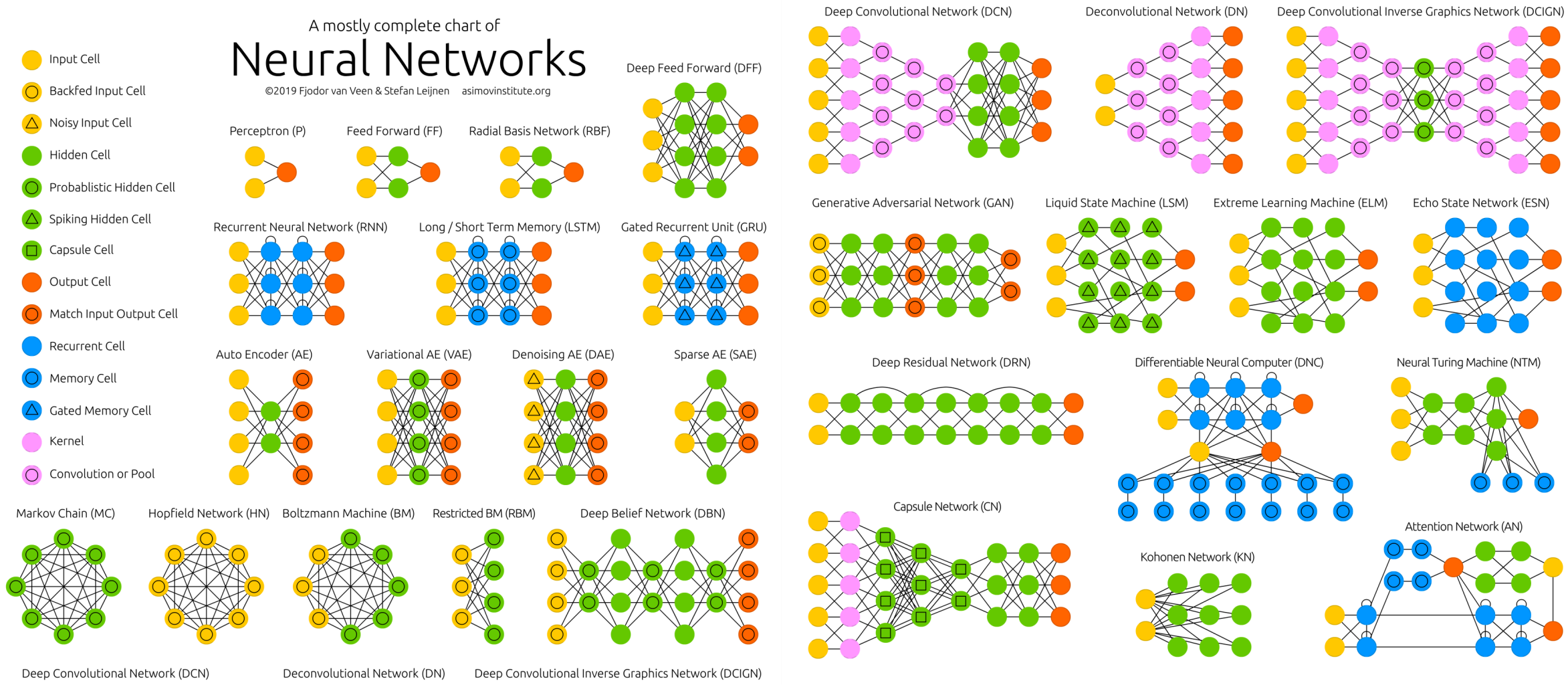
$$\begin{aligned}\nabla L(\theta) &= \nabla\left(\frac{1}{N}\sum_{k=1}^{N}\|f_\theta(x_k)-y_k\|_2^2\right)\\ &= \frac{1}{N}\sum_{k=1}^{N}\nabla\|f_\theta(x_k)-y_k\|_2^2\\ &= \frac{1}{N}\sum_{k=1}^{N}\left(2\|f_\theta(x_k)-y_k\|_2\cdot\nabla f_\theta(x_k)\right).\end{aligned}$$

- This means that we need to *propagate* the error through our model f_θ.
- Since a neural network is a chain of neurons, we need to apply the **chain rule** of differentiation.
- As a consequence the loss is **backpropgated** through the network to determine the individual descent directions: $\frac{\partial L}{\partial \theta_i}$.

> This is called the **backpropagation** algorithm. We require one forward pass and one backward pass for every data tuple (x_k, y_k). Taking the average gives us the average steepest-descent improvement over the dataset $\mathcal{D}_{\text{train}}$ for the current θ.

Regularization

Neural network architectures



The neural network zoo [Asimov Institute].

Linear regression as a special case

Summary / what you have learned

Summary / what you have learned

- a

References

Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Wallscheid, and Daniel Weber. 2026. "Reinforcement Learning: A Comprehensive Open-Source Course." *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:10.21105/jose.00306.

Abu-Mostafa, Yaser S, Malik Magdon-Ismail, and Hsuan-Tien Lin. 2012. *Learning from Data*. Vol. 4. AMLBook New York.

Bengio, Yoshua, Ian Goodfellow, Aaron Courville, et al. 2017. *Deep Learning*. Vol. 1. MIT press Cambridge, MA, USA.

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Vol. 4. 4. Springer.

Hastie, Trevor. 2009. "The Elements of Statistical Learning: Data Mining, Inference, and Prediction." springer.

Li, Hao, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2018. "Visualizing the Loss Landscape of Neural Nets." *Advances in Neural Information Processing Systems* 31.